

Solutions to CS687 Homework 1

Krishna Kumar Bais

Roll No.: 241110038

Problem 1:

Problem Statement

Recall that in the notes, we defined a language A to be decidable if both A and A^c are Turing acceptable.

Show that this is equivalent to saying that a language A is decidable if and only if there is a Turing machine that accepts every string in A and halts, and rejects every string not in A and halts.

Proof

We need to prove the equivalence of the two definitions of decidability:

Definition 1: A language A is decidable if both A and A^c are Turing acceptable.

Definition 2: A language A is decidable if there exists a Turing machine M that accepts every string in A and halts, and rejects every string not in A and halts.

Part 1: Definition 2 Implies Definition 1

Assume that A is decidable according to Definition 2. That is, there exists a Turing machine M such that:

- For all $x \in A$, M halts and accepts x .
- For all $x \notin A$, M halts and rejects x .

We need to show that both A and A^c are Turing acceptable.

Turing Acceptability of A : Since M halts and accepts every $x \in A$, A is Turing acceptable by definition.

Turing Acceptability of A^c : For any $x \notin A$, M halts and rejects x . This means that M halts on every input $x \notin A$, and since M rejects x , we can construct a Turing machine M' that accepts x if and only if M rejects x . Thus, A^c is Turing acceptable.

Hence, if A is decidable according to Definition 2, then both A and A^c are Turing acceptable.

Part 2: Definition 1 Implies Definition 2

Assume that A is decidable according to Definition 1. That is, both A and A^c are Turing acceptable. We need to show that there exists a Turing machine M that halts on every input x and accepts x if $x \in A$, and rejects x if $x \notin A$.

Turing Machines for A and A^c : Since A is Turing acceptable, there exists a Turing machine M_A such that:

- If $x \in A$, then M_A halts and accepts x .
- If $x \notin A$, then M_A either halts and rejects x or does not halt.

Similarly, since A^c is Turing acceptable, there exists a Turing machine M_{A^c} such that:

- If $x \in A^c$, then M_{A^c} halts and accepts x .
- If $x \notin A^c$, then M_{A^c} either halts and rejects x or does not halt.

Constructing the Decider M : We construct a Turing machine M that simulates both M_A and M_{A^c} in parallel (using dovetailing).

For any input x :

- If M_A halts and accepts x , then M halts and accepts x .
- If M_{A^c} halts and accepts x , then M halts and rejects x .

Since A and A^c are Turing acceptable, for every x , either M_A or M_{A^c} will halt and accept x . Thus, M will always halt on every input x .

Correctness of M :

- If $x \in A$, then M_A halts and accepts x , so M halts and accepts x .
- If $x \notin A$, then $x \in A^c$, so M_{A^c} halts and accepts x , and M halts and rejects x .

Thus, M is a decider for A , and A is decidable according to Definition 2.

Conclusion

We have shown that:

- If A is decidable according to Definition 2, then both A and A^c are Turing acceptable (Definition 1).

- If A is decidable according to Definition 1, then there exists a Turing machine M that halts on every input and decides A (Definition 2).

Therefore, the two definitions of decidability are equivalent.

Problem 2:

Prove that every infinite computably enumerable language contains an infinite decidable subset.

Proof

Let A be an infinite computably enumerable language. We construct an infinite decidable subset $B \subseteq A$ as follows:

Construction of B

Enumerate A as $x_1, x_2, x_3, \dots$. Define B by selecting elements in **strictly increasing lexicographical order**:

- Initialize $B = \emptyset$ and set $\text{current_max} = \lambda$ (empty string).
- For each x_i in the enumeration of A :
 - If x_i is lexicographically larger than current_max :
 - * Add x_i to B .
 - * Update $\text{current_max} = x_i$.

Verification

- **Infiniteness:** Since A is infinite and strings are compared lexicographically, there are infinitely many distinct strings in A . The construction ensures B contains infinitely many increasing elements.
- **Decidability:** To decide if $w \in B$:
 1. Enumerate A until encountering a string x_j where $x_j \geq w$ lexicographically.
 2. If $x_j = w$ and x_j was added to B (i.e., $x_j >$ all previous elements), accept. Else, reject.

This process halts because A is infinite, so we will eventually find a string $\geq w$.

Conclusion

The subset B constructed this way is infinite and decidable. Thus, every infinite c.e. language contains an infinite decidable subset.

Problem 3:

Problem Statement

Show that if x is any finite string, and $f : \Sigma^* \rightarrow \Sigma^*$ is any total computable function, then

$$C(f(x)) \leq C(x) + O(1).$$

This inequality states that you can never increase information through computation; you can either preserve it or decrease it.

Solution

We will prove the **Data Processing Inequality** for Kolmogorov complexity. The key idea is that if f is a total computable function, then the complexity of $f(x)$ cannot exceed the complexity of x by more than a constant that depends only on f .

-> Constructing a Program for $f(x)$

Let p be the shortest program that outputs x on the universal Turing machine U , i.e., $U(p) = x$. Since f is a total computable function, there exists a Turing machine M_f that computes f . We can use M_f to construct a program q that outputs $f(x)$ as follows:

1. Program q :

The program q consists of two parts:

- A description of M_f , which is a fixed string of length c_f (where c_f depends only on f).
- The program p , which outputs x .

When q is run on U , it first simulates M_f on x (which is obtained by running p on U), and then outputs $f(x)$.

2. Length of q :

The length of q is $|q| = |p| + c_f$, where c_f is the length of the description of M_f .

-> Applying the Definition of Kolmogorov Complexity

By the definition of Kolmogorov complexity, $C(f(x))$ is the length of the shortest program that outputs $f(x)$. Since q is a program that outputs $f(x)$, we have:

$$C(f(x)) \leq |q| = |p| + c_f.$$

But $|p| = C(x)$, because p is the shortest program that outputs x . Therefore,

$$C(f(x)) \leq C(x) + c_f.$$

Conclusion

The constant c_f depends only on the function f (specifically, on the description of the Turing machine M_f), and not on x . Thus, we have shown that:

$$C(f(x)) \leq C(x) + O(1),$$

where $O(1)$ is a constant that depends only on f .

This proves the **Data Processing Inequality** for Kolmogorov complexity.

Problem 4:

Problem Statement

Imagine that you have a hard drive filled with useful data. You place it on a stick of dynamite and light the fuse. The process of making the dynamite, lighting the fuse, and the explosion are all computable processes. Yet, in a few seconds, the data on the hard drive is transformed into an unrecognizable state, effectively increasing disorder. We explore how this relates to the **Data Processing Inequality**.

Analysis of the Scenario

-> Computability of the Physical Process

The steps involved in destroying the hard drive are:

- Constructing the dynamite (a computable process involving chemistry and physics).
- Lighting the fuse (a computable physical interaction).
- The explosion occurs, reducing the hard drive to debris.

Each of these steps can be described algorithmically in principle. However, the outcome of the explosion introduces randomness into the system, making it impossible to recover the original data.

-> Increase in Disorder and Loss of Information

Before the explosion, the hard drive contained structured data X , which had low entropy and high Kolmogorov complexity. After the explosion, the resulting debris can be represented as a state Z , which has:

- Higher entropy (more randomness and disorder).
- Lower mutual information with the original state X (since reconstructing X from Z is nearly impossible).

The explosion acts as a transformation f , mapping X to Z :

$$Z = f(X).$$

Applying the Data Processing Inequality, we get:

$$I(X; Z) \leq I(X; Y),$$

where Y represents some intermediate state (such as the hard drive being heated before destruction). Since Z is the final state of destruction, its information about X is significantly reduced, leading to:

$$I(X; Z) \approx 0.$$

This signifies that almost all useful data has been lost.

-> Relation to Kolmogorov Complexity

Given that f is a computable transformation, the Data Processing Inequality suggests:

$$C(f(X)) \leq C(X) + O(1).$$

However, after the explosion, the resulting data is mostly random, meaning:

$$C(Z) \gg C(X).$$

This contradicts the idea that computable functions cannot increase complexity. The resolution lies in the fact that while f is computable, the explosion introduces **randomness**, which is incompressible and not generated by a structured program. Thus, the post-explosion state Z has high entropy but low structured information.

Conclusion

The destruction of the hard drive demonstrates the **Data Processing Inequality** in action. The explosion is a computable process, but it irreversibly increases entropy, making data reconstruction impossible. This aligns with the fundamental principle that processing cannot increase information content but can degrade it irreversibly.

Problem 5:

Problem Statement

Let x be an arbitrary finite binary string. Do all permutations of x have the same plain Kolmogorov complexity as x ? If so, prove your claim. Otherwise, construct a counterexample, and prove that your string has some permutation which has a significantly different Kolmogorov complexity.

Analysis: When Do Permutations Have the Same Complexity?

In general, the Kolmogorov complexity of a string depends on its structure and compressibility. The following cases arise:

1. Random Strings: If x is a truly random binary string of length n , then every permutation of x is equally random. Since randomness implies incompressibility, we have:

$$C(x) \approx C(\text{perm}(x)) \approx n.$$

Thus, all permutations have approximately the same Kolmogorov complexity.

2. Highly Structured Strings: If x has a simple repetitive structure (e.g., $x = 000 \dots 111$), then its complexity is low since a short program can generate it. However, some permutations of x may disrupt this pattern and become less compressible, increasing complexity.

Counterexample: Constructing a String with Unequal Permutation Complexities

Consider the string:

$$x = 000 \dots 001111 \dots 11,$$

where the first half consists of all 0s and the second half consists of all 1s.

- The original string has a simple description: "write $n/2$ zeros, then $n/2$ ones," leading to a complexity:

$$C(x) = O(\log n).$$

- A permutation that shuffles 0s and 1s randomly (e.g., "0101100110...") appears random, making its shortest program essentially store the entire string. Thus, its complexity approaches:

$$C(\text{perm}(x)) \approx n.$$

Since $O(\log n)$ and n differ significantly, this proves that permutations can have widely different Kolmogorov complexities.

Conclusion

Not all permutations of a string necessarily have the same Kolmogorov complexity. While the complexity remains similar for random strings, structured strings can have permutations that significantly increase their complexity.

$$\exists x, \exists \text{perm}(x) \text{ such that } |C(x) - C(\text{perm}(x))| = \Theta(n).$$

Thus, the claim that all permutations have the same complexity is false.

Problem 6:

Problem Statement

Let χ_H be the infinite binary sequence where $\chi_H[i] = 1$ if the i -th Turing machine halts, and $\chi_H[i] = 0$ otherwise. Show that for all sufficiently large n :

$$C(\chi_H[0 \dots n-1]) \leq \log n + O(1),$$

where $\chi_H[0 \dots n-1]$ denotes the n -length prefix of χ_H .

Solution

The sequence χ_H encodes the halting problem, which is uncomputable. However, its prefixes can be highly compressible due to their algorithmic structure.

Key Insight

While χ_H itself is uncomputable, its finite prefixes can be compressed using a program that systematically generates the first n bits of χ_H with a halting oracle.

Construction of Compressing Program

We construct a program p_n that outputs $\chi_H[0 \dots n-1]$:

1. Input:

The value n , encoded in $\log n$ bits.

2. Procedure:

- For each $i = 0, 1, \dots, n-1$:
 - Use a halting oracle to determine if the i -th Turing machine halts.
 - Set $\chi_H[i] = 1$ if it halts, otherwise set it to 0.
- Output the resulting n -bit string.

Program Size Analysis

- The fixed part of the program (instructions for generating $\chi_H[0 \dots n-1]$) requires $O(1)$ bits.
- The input n requires $\log n$ bits.
- The total size of the program is therefore:

$$|p_n| = \log n + O(1).$$

Since this program generates the prefix $\chi_H[0 \dots n - 1]$, we have:

$$C(\chi_H[0 \dots n - 1]) \leq |p_n| = \log n + O(1).$$

Conclusion

The prefixes of the characteristic sequence of the halting problem are compressible, and their Kolmogorov complexity satisfies:

$$C(\chi_H[0 \dots n - 1]) \leq \log n + O(1).$$

This demonstrates that even though the sequence is uncomputable, its prefixes can be highly compressible.

Problem 7:

Problem Statement

Let P be a prefix-free set of strings forming the domain of a universal prefix-free machine M . Define Chaitin's Omega as:

$$\omega = \sum_{\substack{p \in P \\ M(p) \downarrow}} \frac{1}{2^{|p|}},$$

where $M(p) \downarrow$ means that $M(p)$ halts. Show that for all sufficiently large n , the prefixes of ω are incompressible:

$$C(\omega[0 \dots n - 1]) \geq n - O(1).$$

The purpose of this question is to show that there are uncomputable strings which are also incompressible, in contrast to the previous question.

Solution

Chaitin's Omega is an uncomputable real number that encodes information about which programs halt. Despite being uncomputable, its prefixes are incompressible.

Key Insight

If a prefix of Chaitin's Omega could be compressed significantly, it would allow us to solve the halting problem for all programs up to a certain length—an impossibility.

Proof of Incompressibility

-> Structure of Chaitin's Omega:

The binary expansion of Chaitin's Omega encodes whether programs halt. Specifically:

- If a program p_i halts, its contribution to Omega ($2^{-|p_i|}$) determines specific bits in its binary expansion.
- The first n -bit prefix of Omega, denoted by $\omega[0 \dots n-1]$, contains information about whether programs with descriptions up to length approximately $n/2 + O(1)$ halt.

-> Contradiction via Compression:

Suppose for contradiction that there exists a program p' of length less than $n - c_0$ (for some constant $c_0 > 0$) that generates the first n -bit prefix of Omega ($\omega[0 \dots n-1]$). Then:

- Using this program, we could determine whether programs up to length approximately $n/2 + O(1)$ halt.
- This would solve the halting problem for all such programs a contradiction since the halting problem is undecidable.

Thus, no such short program can exist.

-> Lower Bound on Complexity:

The shortest program capable of generating the first n -bit prefix of Omega must have length at least:

$$C(\omega[0 \dots n-1]) \geq n - O(1),$$

where the constant term accounts for fixed overhead in describing how to interpret and execute such a program.

Conclusion

The prefixes of Chaitin's Omega are incompressible. For sufficiently large n , their Kolmogorov complexity satisfies:

$$C(\omega[0 \dots n-1]) \geq n - O(1).$$

This demonstrates that there exist uncomputable sequences (like Chaitin's Omega) whose prefixes are also incompressible, contrasting with Problem 6.

Problem 8:

Problem Statement:

Show, using an argument involving plain Kolmogorov complexity, that for all sufficiently large n , most strings of length n will not have a contiguous stretch of more than $\log^2 n$ zeros anywhere in the string.

Key Ideas:

- **Plain Kolmogorov Complexity ($C(x)$):** The length of the shortest program that outputs x
- **Incompressibility:** A string x is incompressible if $C(x) \geq |x| - c$ for some constant c
- **Incompressibility Theorem:** At least $2^n(1 - 2^{-c})$ strings of length n satisfy $C(x) \geq n - c$
- **Compression Argument:** Strings with long zero runs can be significantly compressed

Proof:

Setup

Let $k = \log^2 n$. We demonstrate that strings containing k -length zero runs are compressible and therefore exponentially rare.

Encoding Scheme

For any string x with a k -length zero run:

- Position i of the zero run: $\log n$ bits
- Remaining $n - k$ bits: $n - k$ bits
- Fixed decoder program: $O(1)$ bits

Complexity Bound

The total description length gives:

$$C(x) \leq \underbrace{\log n}_{\text{position}} + \underbrace{n - k}_{\text{remaining bits}} + \underbrace{O(1)}_{\text{program}} = n - \log^2 n + O(\log n)$$

Incompressibility Contradiction

For incompressible strings ($C(x) \geq n - c$):

$$n - \log^2 n + O(\log n) < n - c \quad \text{for } n > 2^{O(c)}$$

This inequality fails dramatically, showing such strings are rare.

Counting Compressible Strings

There are at most:

$$\underbrace{n}_{\text{positions}} \times \underbrace{2^{n-\log^2 n}}_{\text{remaining bits}} = 2^{n-\log^2 n+\log n}$$

strings with $\log^2 n$ -length zero runs, constituting a fraction:

$$\frac{2^{n-\log^2 n+\log n}}{2^n} = 2^{-\log^2 n+\log n} = 2^{-\Omega(\log^2 n)}$$

Conclusion:

The probability that a random string contains a $\log^2 n$ -length zero run is bounded by $2^{-\Omega(\log^2 n)}$. Therefore, for sufficiently large n , most strings avoid such long zero runs with probability approaching 1.